

ILA Tutorial

An Integrated Logic Analyzer (ILA) is a software-based logic analyzer which allows an FPGA to be monitored from a PC. It can be used to verify hardware functionality and debug a running FPGA. The ILA has two main components:

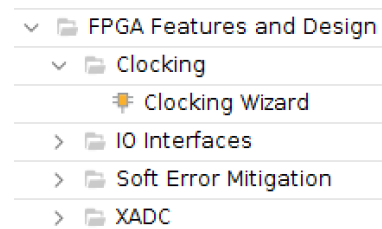
- Debug Hub: connects pins on the FPGA to the Hardware Manager software
- ILA Core: monitors the connectors and does the actual analysis

This tutorial will explain how to examine a version of the PS/2 interface from Exercise 1 using an ILA. It will demonstrate how to generate an ILA core. It will also show how to monitor the running FPGA hardware to capture a make code from the PS/2 keyboard.

Before beginning this tutorial, a Xilinx project should be set up with the provided PS/2 interface VHDL code and constraints file added to it. These files are “PS2_behv.vhd” and “PS2Interface.xdc” and can be found in the Exercise 1 module on MyCourses.

1. Since the source clock of the board is running much faster than the clock domain of the PS/2 interface system, the clock needs to be scaled down for the ILA core to properly sample the interface signals. A **Xilinx Clocking Wizard IP Core** will be generated to divide the 100MHz down to 25MHz. First, the target language of the project needs to change by modifying the respective option under the Project Settings. This will ensure that Vivado generates a VHDL instantiation template. Click on **Settings** under the **Project Manager** section in the Flow Navigator and make sure that the **Target Language** is set to **VHDL**.

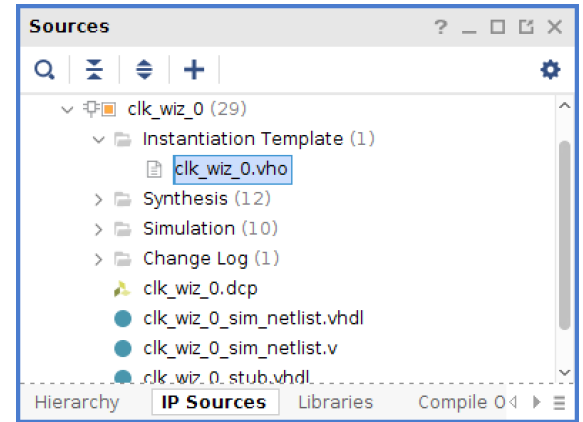
2. To add the IP core to the project, click on **IP Catalog** under the **Project Manager** section in the Flow Navigator. Navigate to **FPGA Features and Design** → **Clocking** and double click the **Clocking Wizard** to start it.



3. Leave the default options under the **Clocking Options** tag and navigate to the **Output Clocks** tab inside the wizard window. Change the clock value under the **Request Output Frequency** column from 100MHz to 25MHz. Deselect the **locked** option under the **Enable Optional Inputs** section at the bottom of the page. Click **OK** and then click **Generate** in the window that appears to allow Vivado to generate the necessary source files.

Board	Clocking Options	Output Clocks	Port Renaming	MMCM S
The phase is calculated relative to the active input clock.				
Output Clock	Port Name	Output Freq (MHz)		
		Requested	Actual	
☒ clk_out1	clk_out1	25.000	25.000	
☐ clk_out2	clk_out2	100.000	N/A	

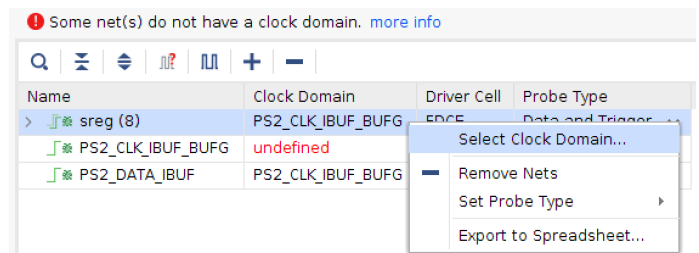
- Now an instance of the new core needs to be instantiated and connected to the proper signals for the desired functionality. Xilinx Core generator creates a component and instance declaration template file to allow a user to easily create an instance of the new generated core. This template file can be found under the **IP Sources** tab in the **Sources** window and then expanding the navigator for new generated clock wizard core. The template file is referenced under the **Instantiation Template** folder. Open the *clk_wiz_0.vho* and copy the component block for the clocking wizard core into the architecture portion of the source. Then, copy the instantiation block and copy it under the begin portion of the architecture. Ensure the design has an input for the main FPGA 100MHz clock called "CLK_1000MHZ" and connect that signal to the *clk_in1* port and create a new signal and connect it to the *clk_out1* port.

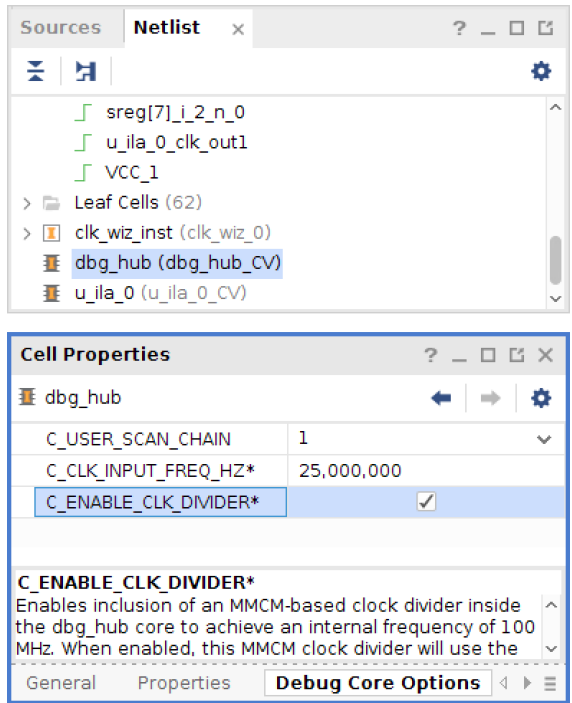


- To mark signals for sample and debug, a Xilinx signal attribute called *mark_debug* can be used to flag to Vivado that these signals should be connected to the ILA core. The PS2_CLK, PS2_DATA, and sreg signals of the design will be flagged for debug. Add the following lines in the middle of the architecture section of the code before the begin statement:

```
-- Debug signal flags
attribute mark_debug : string;
attribute mark_debug of PS2_CLK: signal is "true";
attribute mark_debug of PS2_DATA: signal is "true";
attribute mark_debug of sreg: signal is "true";
```

- Next, Vivado needs to be instructed to generate an ILA core and to connect the debug signals that were marked. To do so, first the design needs to be synthesized. Synthesize the design as normal. After the synthesis is complete, click on **Open Synthesized Design** to open the Synthesis Design window. Next, click on **Tools** at the top of the Project Window and select **Set Up Debug...** from options that appear.
- Click **Next** in the window that appears to navigate to the **Nets to Debug** menu. The signals that were marked for debug should appear in the net list. However, there will be a warning that some nets do not have a clock domain since this implementation does not synchronize the design with the system clock. For purposes of sampling these signals, the clock domain of all the signals will just be set to the divided clock to instruct the core to sample based on this clock. Highlight all the nets in the list and right click the **Select Clock Domain...** option. Select the output clock of the clock wizard instance that is denoted *clk_wiz_inst/inst/clk_out1*. Click **Ok** and then click **Next** to navigate to the next menu.



8. Select a **Sample Data Depth** of **65536** and select the **Capture Control** trigger setting. The sample data depth indicates how many samples the core should record after a trigger occurs. The capture control setting will allow a user to instruct the core on what data conditions the core should trigger and begin sampling the data. Click **Next** and then **Finish** to allow Vivado to generate the new ILA core and update the schematic.
 9. A constraint property needs to be set that instructs the ILA core that the system is running on a slower clock than its default since a 25MHz clock has been connected. Open the netlist window in the opened Synthesis window by clicking on the **Netlist** tab at the top of the **Sources** window section. Select **dbg_hub** in the window and select the **Debug Core Options** tab near the bottom of the window. Click the checkbox next to the **C_ENABLE_CLK_DIVIDER** attribute option and set the **C_CLK_INPUT_FREQ_HZ** attribute option to 25,000,000. Then save the changes made the constraints file by clicking the Save Constraints option under the File menu at the top of the Project window or by clicking Ctrl+S. To ensure that these changes took effect, check the constraints file and observe that the changes to the attributes were saved. Uncomment the lines for the external clock input.
- 
- The screenshot shows two windows from the Vivado IDE. The top window is the 'Sources' window with the 'Netlist' tab selected, displaying a list of components including 'sreg[7]_i_2_n_0', 'u_ila_0_clk_out1', 'VCC_1', and 'dbg_hub (dbg_hub_CV)'. The bottom window is the 'Cell Properties' window for the 'dbg_hub' component, with the 'Debug Core Options' tab selected. It shows attributes 'C_USER_SCAN_CHAIN' set to 1, 'C_CLK_INPUT_FREQ_HZ*' set to 25,000,000, and 'C_ENABLE_CLK_DIVIDER*' checked. A description for 'C_ENABLE_CLK_DIVIDER*' is visible at the bottom: 'Enables inclusion of an MMCM-based clock divider inside the dbg_hub core to achieve an internal frequency of 100 MHz. When enabled, this MMCM clock divider will use the'.
10. Finish the implementation process as normal and generate a new bitstream for the system.
 11. After a bitstream has been generated, program the board with the file as normal. Observe that alongside the new bitstream is debug probe description file denoted with an .ltx extension. This file describes the ports that have been connected the ILA core. Click **Program** to program the board and bring up hardware ILA status windows.
 12. The hardware ILA status windows are used to observe the debug signals and command the debug core how it should operate. The following windows are displayed for to the user:
 - Settings Window: Allows the user to set some settings in accordance with how the data is captured.
 - The Status Window: Display to the user the status of the core with some controls to arm the core with a new trigger.
 - Trigger Setup Window: Allows a user control the signal conditions on which the debug core will trigger and capture the data.
 - Capture Setup Window: Allows a user to control the conditions that the debug should capture. This option usually allows a user more options for capture.
 - Waveform Window: Displays the user the waveforms of the debug signals after a capture is read from the core. The waveforms are displayed in a similar way to that of a simulation waveform. Note that the scale of the waveform is in the number of ILA core clock periods.

13. The debug core trigger needs to be set up to capture the status of the signals when a break code has been detected as desired. This condition will be met when the **sreg** value is set to the break code byte value of hexadecimal F0. However, it is convenient to observe the data and clock signals beforehand to observe the data being transferred. With these conditions in mind, a trigger will be set up that will capture the signals after the byte value F0 is detected and to record several samples before the trigger occurs. In the Settings window, set the **Trigger Position in Window** option to **20,000**. This instructs the core to capture 20,000 samples before the trigger occurs. Add a new trigger in the Trigger Setup window by clicking the blue plus  icon and selecting the **sreg** signal from the menu that appears. Set the operator option to **==** and the value option to **F0**. This instructs the core to trigger and capture after the value of **sreg** is equal to **F0**.
14. Next the debug core needs to be armed with the trigger that was set up. To arm the core, click the **Run Trigger for this ILA core**  button in status window. If the trigger properly arms, the core status should highlight **Waiting to Trigger** assuming the trigger was not detected as soon as the core was armed.
15. Capture a break code and make code combination in the waveform window by pressing and releasing a key on the connected keyboard to trigger the debug core. Record the results if the waveform displays signal values as expected.